Corso di Python

Lezione 2

Variabili, cicli, funzioni e dizionari

Andrea Tosi / 11 novembre 2026

Argomenti della lezione

- Variabili, tipi di dati e condizioni
- Liste, dizionari, cicli e funzioni
- Un progetto: preparare la lista della spesa

Verificare la versione di Python

Terminale Ubuntu / macOS

```
python3.13 --version
```

- Ubuntu 26.04 (anche in WSL) o macOS
- Python 3.13.18 della guida
- Terminale integrato di VS Code

RISULTATO

```
Python 3.13.18
```

Creare ed eseguire un file Python

Python

```
print("Milo ha fame. Di nuovo.")
```

- Salva il codice in lezione2.py
- Ctrl+S / Cmd+S per salvare
- Apri il terminale nella cartella del file

RISULTATO

```
Milo ha fame. Di nuovo.
```

Ogni esempio è indipendente, salvo indicazione diversa.

Terminale: python3.13 lezione2.py

Variabili

Python

```
numero_limoni = 3
print(numero_limoni)

numero_limoni = numero_limoni + 1
print(numero_limoni)
```

- Un nome fa riferimento a un valore
- Si calcola prima la parte a destra
- Poi si aggiorna il nome a sinistra

RISULTATO

```
3
4
```

I nomi distinguono maiuscole e minuscole. Non possono iniziare con una cifra.

Parole riservate

Queste 35 parole hanno un significato nella sintassi: non puoi usarle come nomi di variabili o funzioni.

```
False      await      else       import     pass
None       break      except     in         raise
True       class      finally    is         return
and        continue   for        lambda     try
as         def        from       nonlocal   while
assert     del        global     not        with
async      elif       if         or         yield
```

match, case, _ e type sono riservate solo in alcuni contesti.

print, len, list e str sono nomi predefiniti: evita di riassegnarli.

Per esempio, for = 3 genera SyntaxError.

Tipi di dati

Python

```
print(type(4))
print(type(0.8))
print(type("Milo"))
print(type(True))
```

- int: interi, float: decimali
- str: testo tra virgolette
- bool: True oppure False

RISULTATO

```
<class 'int'>
<class 'float'>
<class 'str'>
<class 'bool'>
```

type(valore) indica il tipo. True e False non hanno virgolette.

Operatori aritmetici

Python

```
print(7 + 2, 7 - 2)
print(7 * 2, 7 / 2)
print(7 // 2, 7 % 2)
print(2 ** 3)
print((7 + 2) * 3)
```

- + - * /: le operazioni usuali
- //: quoziente arrotondato verso il basso
- %: resto, **: potenza

RISULTATO

```
9 5
14 3.5
3 1
8
27
```

print(7 + 2, 7 - 2) stampa due valori sulla stessa riga.

Stringhe e conversioni

Python

```
frutto = "limone"
quantita = int("3")

print(quantita + 2)
print(f"{quantita} {frutto}. Che italiano!")
```

- "3" è testo, 3 è un numero
- int() converte un testo numerico
- f"..." inserisce valori tra graffe

RISULTATO

```
5
3 limone. Che italiano!
```

Confronti e operatori logici

Python

```
limoni = 4

print(limoni >= 4)
print(limoni == 0)
print(limoni > 0 and limoni < 10)
print(limoni == 0 or limoni == 4)
print(not (limoni > 0))
```

- = assegna, == confronta
- Un confronto produce True o False
- and, or e not combinano condizioni

RISULTATO

```
True
False
True
True
False
```

Confronti: == uguale, != diverso, < minore, <=, > maggiore, >=

Istruzioni condizionali

Python

```
frutti = 4

if frutti >= 3:
    print("Macedonia!")
elif frutti > 0:
    print("Una merenda triste.")
else:
    print("Il frigo fa eco.")
```

RISULTATO

```
Macedonia!
```

- I due punti aprono il blocco
- Il rientro è di quattro spazi
- Si esegue il primo ramo vero

Nomi, commenti e rientri

Python

```
PREZZO_LIMONE = 30  # centesimi
numero_limoni = 2

if numero_limoni > 0:
    totale = numero_limoni * PREZZO_LIMONE
    print(totale)
```

- Variabili minuscole, COSTANTI maiuscole
- # introduce un commento fino a fine riga
- 4 spazi per rientro. Non mescolare spazi e tab.

RISULTATO

```
60
```

In VS Code imposta “Spaces: 4”: il tasto Tab deve inserire spazi.

Le maiuscole sono una convenzione: Python permette di riassegnare una costante.

Liste e indici

Python

```
macedonia = ["mela", "banana", "sedia"]

print(macedonia[0])
print(macedonia[-1])
print(len(macedonia))
print("kiwi" in macedonia)
```

- Il primo indice è 0, l'ultimo è -1
- len() conta gli elementi
- in verifica la presenza di un valore

RISULTATO

```
mela
sedia
3
False
```

Modificare una lista

Python

```
macedonia = ["mela", "banana", "sedia"]
macedonia[2] = "kiwi"
macedonia.append("limone")

print(macedonia)
```

- L’indice sceglie cosa sostituire
- append() aggiunge in fondo
- La lista cambia sul posto

RISULTATO

```
['mela', 'banana', 'kiwi', 'limone']
```

append() modifica la lista e restituisce None.

Il ciclo for

Python

```
macedonia = ["mela", "banana", "kiwi"]

for frutto in macedonia:
    print(f"Milo annusa: {frutto}")

print("Milo voleva il tonno.")
```

- frutto riceve ogni elemento
- Il blocco si ripete per ciascuno
- Fuori dal rientro, il ciclo è finito

RISULTATO

```
Milo annusa: mela
Milo annusa: banana
Milo annusa: kiwi
Milo voleva il tonno.
```

Calcolare un totale

Python

```
prezzi = [50, 40, 30]
totale = 0

for prezzo in prezzi:
    totale = totale + prezzo

print(totale)
```

- Il totale parte da zero
- Ogni giro aggiunge una spesa
- Il risultato si stampa alla fine

RISULTATO

```
120
```

Prezzi in centesimi: 50 + 40 + 30 = 120

Funzioni e parametri

Python

```
def costo(quantita, prezzo):
    return quantita * prezzo

print(costo(3, 50))
print(costo(2, 40))
```

- def definisce la funzione
- I parametri ricevono gli argomenti
- return restituisce il risultato

RISULTATO

```
150
80
```

La differenza tra print e return

Python

```
def saluta(nome):
    print(f"Ciao {nome}!")

risultato = saluta("Milo")
print(risultato)
```

- print mostra il valore
- return permette di riutilizzarlo
- Senza return si ottiene None

RISULTATO

```
Ciao Milo!
None
```

Condizioni e return nelle funzioni

Python

```
def biglietti_mancanti(ospiti, biglietti):
    differenza = ospiti - biglietti
    if differenza > 0:
        return differenza
    return 0

print(biglietti_mancanti(5, 2))
print(biglietti_mancanti(2, 5))
```

RISULTATO

```
3
0
```

- Può contenere condizioni
- return termina la chiamata
- Un risultato negativo può non avere senso

Le variabili di una funzione

Python

```
limoni = 10

def prepara(limoni):
    porzioni = limoni + 1
    return porzioni

print(prepara(2))
print(limoni)
```

RISULTATO

```
3
10
```

- Lo scope indica dove un nome è accessibile
- Parametri e variabili assegnate sono locali
- Il nome locale non cambia quello esterno

Fuori dalla funzione, print(porzioni) genera NameError.

Le variabili dopo if e for

Python

```
if True:
    messaggio = "Macedonia!"
else:
    messaggio = "Solo limoni."
for frutto in ["mela", "kiwi"]:
    ultimo = frutto
print(messaggio)
print(ultimo, frutto)
```

- if, else e for non creano uno scope separato
- I nomi restano nello scope che contiene il blocco
- La variabile del ciclo conserva l'ultimo valore

RISULTATO

```
Macedonia!
kiwi kiwi
```

Un ramo saltato o un ciclo vuoto non eseguono le assegnazioni al loro interno.

Dizionari

Python

```
scatola = {
    "contenuto": "limone",
    "quantita": 3
}

print(scatola["contenuto"])
print(scatola["quantita"])
```

- Le graffe racchiudono le coppie
- La chiave sceglie il valore
- I due punti separano chiave e valore

RISULTATO

```
limone
3
```

Aggiungere e modificare una voce

Python

```
scatola = {"contenuto": "limone"}
scatola["contenuto"] = "Milo"
scatola["umore"] = "offeso"

print(scatola["contenuto"])
print(scatola["umore"])
```

- Una chiave esistente aggiorna il valore
- Una chiave nuova aggiunge una voce
- Le altre voci restano invariate

RISULTATO

```
Milo
offeso
```

Leggere una chiave con get()

Python

```
frutta = {"mele": 2, "banane": 0}

print(frutta.get("mele", "assente"))
print(frutta.get("banane", "assente"))
print(frutta.get("kiwi", "assente"))
print(frutta.get("kiwi"))
print("kiwi" in frutta)
```

- Se la chiave esiste, restituisce il suo valore
- Se manca, usa il secondo argomento
- La lettura non aggiunge una nuova voce

RISULTATO

```
2
0
assente
None
False
```

Senza secondo argomento, get() usa None. frutta["kiwi"] genera invece KeyError.

Iterare su un dizionario

Python

```
ordine = {"caffe": 2, "brioche": 0}

for nome in ordine:
    print(nome, ordine[nome])
```

- Il ciclo riceve una chiave alla volta
- Le parentesi quadre leggono il valore
- Vale anche per un dizionario vuoto

RISULTATO

```
caffe 2
brioche 0
```

Leggere e correggere un errore

Python

```
limoni = "3"
print(limoni + 2)
```

- L’ultima riga dell’errore dà un indizio
- Controlla nomi, chiavi e rientri
- Un programma che parte può dare risultati sbagliati

ERRORE INTENZIONALE

`TypeError`

Correzione: print(int(limoni) + 2)

Esempio completo: l’ordine al bar

Python

```
def costo(quantita, prezzo):
    return quantita * prezzo

ordine = {"caffe": 2, "brioche": 0}
prezzi = {"caffe": 120, "brioche": 150}

scontrino = {}
totale = 0
```

- costo() calcola il prezzo di più pezzi
- I dizionari contengono quantità e prezzi
- {} crea il dizionario vuoto dello scontrino

Primo blocco: continua con il codice delle due slide successive.

Esempio completo: lo scontrino

Python

```
for nome in ordine:
    q = ordine[nome]
    if q > 0:
        scontrino[nome] = q
        totale = totale + costo(q, prezzi[nome])

print(scontrino)
print(totale)
```

- Un ciclo visita i prodotti
- if esclude le quantità uguali a zero
- La funzione contribuisce al totale

RISULTATO

```
{'caffe': 2}
240
```

Continua nello stesso file, sotto il blocco precedente.

Esempio completo: il budget

Python

```
budget = 240

if totale <= budget:
    print("Ci stiamo!")
else:
    print("Milo deve offrire.")
```

- <= comprende il caso di uguaglianza
- Il confronto avviene dopo il ciclo
- La regola va provata sulla soglia

OUTPUT DEL PROGRAMMA COMPLETO

```
{'caffe': 2}
240
Ci stiamo!
```

Aggiungi queste righe dopo lo scontrino.

Progetto: la spesa per una merenda

Vuoi preparare una macedonia e una limonata per una merenda tra amici. Ti servono, in tutto, 4 mele, 2 banane e 2 limoni.

Prima di uscire controlli cosa hai già in casa: trovi 1 mela, 3 banane e nessun limone.

Scrivi una funzione che restituisca cosa devi comprare. Il codice già preparato userà il suo risultato per calcolare il costo e controllare il budget.

Dati iniziali

```
ingredienti = ["mele", "banane", "limoni"]
necessari = {"mele": 4, "banane": 2, "limoni": 2}
in_casa = {"mele": 1, "banane": 3, "limoni": 0}
prezzi = {"mele": 50, "banane": 40, "limoni": 30}
budget = 300
```

necessari indica i pezzi richiesti in totale. in_casa indica quelli già disponibili. Ogni ingrediente compare nei tre dizionari.

La funzione riceve ingredienti, necessari e in_casa. Prezzi e budget servono al codice successivo. I dati sono validi e gli importi sono in centesimi.

Risultato atteso

La funzione deve restituire il dizionario {"mele": 3, "limoni": 2}. Le banane bastano già e non devono comparire nel risultato.

Il codice in fondo al file chiamerà la funzione e userà il dizionario per calcolare il costo. Eseguendo tutto il file, otterrai queste righe:

```
Lista della spesa: {'mele': 3, 'limoni': 2}
Costo: 210 centesimi
Budget sufficiente
```

La funzione da scrivere

Scrivi crea_lista_spesa(ingredienti, necessari, in_casa). Deve restituire un nuovo dizionario con soltanto gli ingredienti mancanti e le quantità da comprare.

Usa i parametri ricevuti. Se un ingrediente basta o avanza, non aggiungerlo. Se hai già tutto, oppure la lista è vuota, restituisci {}.

La funzione non deve stampare né modificare i dati di partenza. Il codice già fornito userà il risultato per calcolare la spesa e confrontarla con il budget.

Scrivi la funzione sopra il blocco di utilizzo, poi decommenta quel blocco.

Usare il risultato della funzione

Python

```
lista_spesa = crea_lista_spesa(
    ingredienti, necessari, in_casa
)
print(lista_spesa)
```

- return consegna il dizionario al chiamante
- lista_spesa conserva il risultato
- Il codice successivo lo usa per calcolare il costo

RISULTATO

```
{'mele': 3, 'limoni': 2}
```

Queste righe funzionano dopo che hai scritto la funzione.

Prove sul programma

Con i dati iniziali la funzione deve restituire 3 mele e 2 limoni. Il codice fornito calcolerà 210 centesimi. Cambiando il budget, la lista deve restare uguale.

Con 10 pezzi di ogni ingrediente in casa, il risultato è {}. Con tutte le scorte a zero, il risultato è {"mele": 4, "banane": 2, "limoni": 2}.

Segui i commenti in spesa.py. Decommenta solo il blocco di utilizzo e mantieni i suoi rientri. Prova poi una seconda chiamata con quantità diverse.

Terminale, nella cartella del file: python3.13 spesa.py